

Services & HttpClient

A Service is where the app's real work happens; **HttpClient** is Angular's built-in tool for talking to a backend from inside one. This guide covers why services exist, how HttpClient talks to a real API end to end, and the interceptors and error handling that hold it together.

1 · WHY SERVICES EXIST

Separation of concerns — the restaurant analogy

The **Waiter** (Component): takes your order, brings food to your table, never cooks anything. The **Kitchen** (Service): prepares the actual food — the real work happens here. If the waiter had to cook *and* serve *and* take payments, chaos would follow for every table. Same with Angular: if a Component fetches data, transforms it, validates it, and renders it, it becomes impossible to maintain or reuse. A Service lets many "waiters" (components) use the same "kitchen" (logic) without duplicating it.

COMPONENT'S JOB	SERVICE'S JOB
Render the UI (HTML + bindings)	Call APIs / talk to the backend
Handle user events (click, submit, input)	Business logic & calculations
Show loading / success / error states	Share data between components
Delegate real work to a Service	Cache & manage application state

2 · WHEN SHOULD I CREATE A SERVICE?

A decision guide

CREATE A SERVICE WHEN...	KEEP IT OUT OF A SERVICE
Logic is needed by more than one component	Anything related to rendering HTML
You're calling an API / backend	DOM manipulation
You need to share state across the app	Component-only UI state (e.g. "is this dropdown open")
You have business rules or calculations	Template-specific formatting logic
You want the logic to be unit-testable	

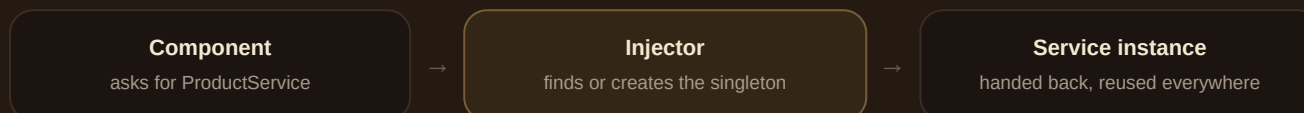
A simple service, before touching HTTP

```
@Injectable({ providedIn: 'root' })
export class CounterService {
  private total = 0;
  increment(): number { return ++this.total; }
  getTotal(): number { return this.total; }
}
// any component can inject CounterService and
// share the SAME "total" value — that's the point.
```

providedIn: 'root' — one shared instance

```
@Injectable({ providedIn: 'root' })
export class ProductService {
  private http = inject(HttpClient);
}
```

`@Injectable()` marks a class as available to Angular's Dependency Injection system — without it, Angular doesn't know how to construct the class. `providedIn: 'root'` registers the service with the app's root injector, making it a **singleton**: created **once**, the first time something injects it, then that same instance is reused everywhere for the lifetime of the app. It's destroyed only when the whole application is destroyed. Bonus: tree-shakable — if nothing ever injects it, it isn't included in the final bundle.



4 · WHY HTTPCLIENT?

Angular's built-in tool for talking to a backend

Why not just use the browser's native `fetch()`? `HttpClient` adds typed responses, Observables (composable, cancellable), interceptors, easy error handling, and testing utilities. Every real app needs to **read** data from a server (products, users, orders) and **write** data back (create, update, delete) — `HttpClient` standardizes this into one consistent API for every HTTP verb, headers, params, and body.

A REQUEST HAS...	A RESPONSE HAS...
A URL — the endpoint address	A status code — 200 OK, 201 Created, 404 Not Found, 500 Server Error
A method — GET / POST / PUT / PATCH / DELETE	Headers — metadata about the response
Headers — metadata (auth token, content-type)	A body — the actual JSON payload
Query parameters — <code>?limit=10&skip=20</code>	
A body — data sent with POST/PUT/PATCH	



The full service implementation

```
@Injectable({ providedIn: 'root' })
export class ProductService {
  private http = inject(HttpClient);
  private baseUrl = 'https://dummyjson.com/products';

  getProducts() {
    return this.http.get<ProductsResponse>(this.baseUrl);
  }
}
```

- `@Injectable({ providedIn: 'root' })` — makes `ProductService` a singleton, available app-wide.
- `private http = inject(HttpClient)` — asks the Injector for the already-configured `HttpClient` singleton.
- `getProducts()` does **not** call `.subscribe()` itself — it hands back a "recipe"; the caller decides when to run it.
- `http.get<ProductsResponse>(url)` — the generic type is a compile-time promise to TypeScript, not a runtime check. Angular does not validate the JSON shape — keep your interfaces accurate.

POST · PUT · DELETE, path vs. query params

```
addProduct(p: Partial<Product>) {
  return this.http.post<Product>(this.baseUrl + '/add', p);
}
updateProduct(id: number, changes: Partial<Product>) {
  return this.http.put<Product>(`${this.baseUrl}/${id}`, changes);
}
deleteProduct(id: number) {
  return this.http.delete(`${this.baseUrl}/${id}`);
}
```

Path parameter — identifies ONE resource

```
// GET /products/10
getProductById(id: number) {
  return this.http.get<Product>(
    `${this.baseUrl}/${id}`
  );
}
```

Query params — filter / paginate

```
// GET /products?limit=10&skip=20
getProducts(limit: number, skip: number) {
  const params = new HttpParams()
    .set('limit', limit).set('skip', skip);
  return this.http.get(this.baseUrl, { params
  });
}
```

`HttpParams` builds and encodes the query string safely (special characters, arrays, spaces) — manual string concatenation like `url + '?limit=' + limit` is error-prone and easy to break.

Sending metadata like auth tokens

```
import { HttpHeaders } from '@angular/common/http';

getSecretData(token: string) {
  const headers = new HttpHeaders({
    Authorization: `Bearer ${token}`,
    'Content-Type': 'application/json',
  });
  return this.http.get('https://dummyjson.com/auth/me', { headers });
}
```

Most real APIs require a token to prove "who is calling" — usually a JWT obtained at login, sent as **Authorization: Bearer <token>** on every protected request. Manually adding this header to **every** service method would be repetitive and error-prone — the clean solution is an HTTP Interceptor that attaches it automatically.

8 · INTERCEPTORS

One place for logic that runs on every request

```
export const authInterceptor: HttpInterceptorFn = (req, next) => {
  const token = localStorage.getItem('token');
  const cloned = token
    ? req.clone({ setHeaders: { Authorization: `Bearer ${token}` } })
    : req;
  return next(cloned);
};

// app.config.ts
providers: [ provideHttpClient(withInterceptors([authInterceptor])) ]
```

An interceptor sits between your app and the network, and can inspect or modify **every** request and response. Common uses: attaching auth tokens automatically, logging requests, global error handling (auto-logout on 401), a global loading indicator, adding common headers — without repeating the logic in every service method.

9 · HANDLING FAILURES

catchError vs. retry

CATCHERROR	RETRY
Use when the error is final — show a message, log it, or fall back to a default	Use when the failure might be transient — a flaky connection, a momentary server hiccup
Recovers or reports, once	Tries the same failed request again, a set number of times

```
return next(req).pipe(
  retry({ count: 2, delay: 500 }),      // try transient failures again
  catchError((error) => {              // then give up gracefully
    console.error('Request failed:', error.message);
    return throwError(() => error);
  }),
);
```

Order matters: **retry** closest to the source, **catchError** after it — so a request gets its second chance before you decide it's truly failed.

 COMMON MISTAKES

API calls directly inside components

Using **any** everywhereNo error handling on `subscribe()`

No loading state — UI looks frozen

Duplicating the same API logic in many places

Hardcoding URLs everywhere

Creating a new service for every tiny thing

 BETTER ARCHITECTURE

Component → Service → HttpClient → API

Typed interfaces for every response

Centralized error handling + interceptors

Explicit loading / error / empty / success states

One service owns one feature's API logic

Base URLs in environment files

Services grouped by domain (Product, User, Order)